

Relatório de Elaboração do Projeto

Next Point — Loja Virtual de Artigos de Padel

Data: 08 de setembro de 2026

Autor: Flávio Saldanha

UFCD 5425

Sumário

1. Introdução
2. Descrição do Projeto
3. Levantamento de Requisitos
4. Análise de Sistemas
5. Design do Sistema
6. Implementação
7. Testes
8. Implantação
9. Conclusão
10. Anexos

1. Introdução

Contextualização

Este relatório documenta o desenvolvimento do projeto "Next Point", uma loja virtual de artigos de padel, cujo objetivo é substituir a gestão informal de vendas através do WhatsApp por uma solução profissional — permitindo aos clientes consultar o catálogo e efetuar encomendas online, e ao lojista gerir produtos, stock e encomendas num único sistema.

Objetivo do Relatório

O objetivo deste relatório é apresentar as etapas de elaboração do projeto, desde o levantamento de requisitos até à implementação, aos testes e à implantação, documentando o progresso e as decisões tomadas — incluindo a mudança de tecnologia ocorrida a meio do desenvolvimento.

2. Descrição do Projeto

Visão Geral

O Next Point é uma aplicação web, desenvolvida em Flask e SQLite, que automatiza a gestão e a venda de artigos de padel, disponibilizando um catálogo público para os clientes e uma área de administração para o lojista.

Âmbito (Escopo)

Incluso: catálogo de produtos, carrinho de compras, checkout, gestão de produtos/categorias/clientes, gestão de encomendas, relatórios de vendas, upload de fotografias.

Não incluso: autenticação de utilizadores, pagamento online, notificações automáticas por email, gestão de fornecedores e integração com APIs de faturação.

Objetivos

- Facilitar a consulta do catálogo e a compra por parte dos clientes, sem depender da troca manual de mensagens.
- Automatizar o controlo de stock e o registo de encomendas.
- Fornecer ao lojista relatórios sobre vendas e produtos mais vendidos.

3. Levantamento de Requisitos

Metodologia de recolha

Os requisitos foram recolhidos a partir de conversas com o dono da loja (o cliente real deste projeto), da análise do processo de vendas atualmente realizado por WhatsApp, e da elaboração de um levantamento formal de requisitos funcionais e não funcionais, entregue como documento próprio.

Requisitos Funcionais (principais)

1. Gestão de produtos, categorias e clientes (criar, consultar, editar, remover).
2. Catálogo online com filtros de pesquisa por categoria e tamanho.
3. Carrinho de compras e finalização de encomendas (checkout).
4. Controlo e atualização automática de stock.
5. Gestão do estado das encomendas (pendente, confirmada, enviada, concluída).
6. Upload de fotografias de produtos, a partir do computador.

Requisitos Não Funcionais (principais)

1. Aplicação desenvolvida em Python 3, com o micro-framework Flask.
2. Persistência de dados em SQLite.
3. Interface simples e responsiva, com Bootstrap.
4. Arquitetura modular, com separação entre dados, lógica de negócio e rotas.
5. Compatibilidade com qualquer sistema operativo com browser.

O levantamento completo de requisitos, com prioridades atribuídas a cada um (RF1–RF20 e RNF1–RNF14), está disponível no documento *Requisitos_NEXTPOINT_FS.pdf*, entregue em anexo.

4. Análise de Sistemas

Modelagem

A modelagem do sistema assenta sobretudo no Diagrama de Entidade-Relacionamento da base de dados, composto por 5 tabelas: categorias, produtos, clientes, encomendas e itens_encomenda. Uma categoria agrupa vários produtos (1:N); um cliente pode ter várias encomendas (1:N); cada encomenda é composta por vários itens (1:N), e cada item refere um produto específico. O diagrama completo, com os relacionamentos e a descrição de cada entidade, está detalhado no documento *workflow_entidades_flavio_saldanha.pdf*, em anexo.

Casos de Uso

1. Consultar Catálogo — o cliente visualiza e filtra os produtos disponíveis.
2. Adicionar ao Carrinho — o cliente escolhe produtos e respetivas quantidades.
3. Finalizar Encomenda — o cliente submete os seus dados e confirma a compra.
4. Gerir Produtos — o administrador cria, edita ou remove produtos e categorias.
5. Gerir Encomendas — o administrador atualiza o estado ou cancela encomendas.
6. Consultar Relatórios — o administrador vê a receita total e os produtos mais vendidos.

Análise de Riscos

Risco	Mitigação
Concorrência no stock — dois clientes a comprar o último artigo em simultâneo.	Validação do stock dentro de uma transação; se algo falhar, é feito rollback automático e nada fica gravado a meio.
Acesso não autorizado à área de administração (sem autenticação nesta versão).	Identificado como funcionalidade prioritária para a próxima versão; documentado como limitação conhecida.
Perda de dados.	Toda a base de dados fica num único ficheiro SQLite, facilmente copiável para cópia de segurança.
Dificuldade de uso por parte do lojista.	Interface simples, construída com Bootstrap, com formulários organizados em janelas modais.

5. Design do Sistema

Arquitetura do Sistema

O sistema segue uma arquitetura em camadas: Interface Web (templates HTML e Bootstrap), Rotas (blueprints Flask), Lógica de Negócio (gestor.py) e Base de Dados (SQLite). As rotas nunca acedem diretamente à base de dados — comunicam sempre com a camada de lógica de negócio, que por sua vez é a única a falar com a base de dados.

Design de Componentes

- **Interface do Utilizador** — desenvolvida com Flask, Jinja2 e Bootstrap 5, dividida em área de loja (cliente) e área de administração (lojista).
- **Lógica de Negócio** — implementada em gestor.py, com uma classe por entidade (GestorProdutos, GestorClientes, GestorEncomendas, GestorCategorias).
- **Base de Dados** — SQLite, gerida por database.py.

Interface do Utilizador — principais ecrãs

- Catálogo de produtos, com filtros de pesquisa
- Ficha de produto (detalhe, foto, preço e stock)
- Carrinho de compras e checkout
- Dashboard de administração
- Gestão de produtos, categorias, clientes e encomendas
- Relatórios e exportação de dados

Base de Dados — Modelo ERD

Tabela	Campos principais
categorias	id, nome
produtos	id, nome, descrição, categoria_id (FK), tamanho, cor, preço, stock, imagem_url
clientes	id, nome, email, telefone
encomendas	id, cliente_id (FK), estado, data_criação, total
itens_encomenda	id, encomenda_id (FK), produto_id (FK), quantidade, preço_unitário

6. Implementação

Plano de Implementação

- **Fase 1** — Preparação do ambiente e criação da base de dados.
- **Fase 2** — Catálogo (categorias e produtos).
- **Fase 3** — Carrinho de compras e encomendas.
- **Fase 4** — Área de administração.
- **Fase 5** — Testes e implantação.

Tecnologias Utilizadas

- Linguagem de Programação: Python 3
- Framework: Flask
- Base de Dados: SQLite
- Frontend: HTML5, CSS3, Bootstrap 5, Jinja2
- Ferramentas: Visual Studio Code, Git, GitHub, DB Browser for SQLite

Estrutura de Código

Ficheiro	Responsabilidade
app.py	Ponto de entrada do sistema; cria a aplicação Flask e liga tudo.
database.py	Ligação à base de dados SQLite e criação do esquema (tabelas).
models.py	Classes de dados (Produto, Cliente, Encomenda...) e exceções do projeto.
gestor.py	Lógica de negócio — regras de stock, cálculos, validações e relatórios.
blueprints/loja.py	Rotas do lado do cliente: catálogo, carrinho, checkout.
blueprints/admin.py	Rotas da área de administração: produtos, categorias, encomendas.

Controlo de Versão

Ferramenta utilizada: Git. **Repositório:** GitHub — github.com/16bitsvnm66/Next.Point.

7. Testes

Plano de Testes

- **Testes Unitários** — validações dos modelos (Produto, Cliente, Categoria).
- **Testes de Integração** — regras de negócio: encomendas, stock, cancelamentos, relatórios.
- **Testes de Sistema** — comportamento ponta-a-ponta das rotas, através do Flask test client.
- **Testes de Aceitação** — validação com o utilizador final (o dono da loja).

Casos de Teste (exemplos)

1. Teste de criação de produto — verificar se um produto é criado corretamente na base de dados.
2. Teste de validação de stock — verificar se uma encomenda é recusada quando não há stock suficiente.
3. Teste de criação de encomenda — verificar se o total é calculado corretamente e o stock é atualizado.
4. Teste de checkout — verificar se o fluxo completo (catálogo → carrinho → checkout) funciona ponta-a-ponta.

Resultados dos Testes

Tipo de teste	Nº de testes	Resultado
Testes unitários	11	Sucesso
Testes de integração	8	Sucesso
Testes de rotas (sistema)	7	Sucesso
Total	26	26/26 — Sucesso

Adicionalmente, foram realizados testes manuais de sistema a todas as rotas da aplicação (catálogo, ficha de produto, carrinho, checkout, administração, relatórios e exportação CSV), num ambiente real com o Flask em modo de desenvolvimento.

8. Implantação

Plano de Implantação

Atividades: configuração do ambiente de produção, migração da base de dados, publicação da aplicação, configuração de domínio e certificado SSL, e monitorização inicial.

Formação

- Sessão de demonstração ao dono da loja, mostrando o catálogo e a área de administração.
- Material de apoio: documento README, com instruções de instalação e utilização.

Suporte

O código-fonte e a documentação ficam disponíveis no repositório GitHub do projeto, permitindo consulta e manutenção futura.

9. Conclusão

Resumo dos Resultados

O Next Point foi desenvolvido em conformidade com os requisitos especificados, com todas as funcionalidades principais implementadas e testadas com sucesso — o Product Backlog está concluído a 100% (39 de 39 tarefas).

Próximos Passos

- Autenticação de utilizadores na área de administração.
- Pagamento online (ex.: MB WAY ou Stripe).
- Notificações automáticas por email sobre o estado da encomenda.
- Gestão de fornecedores e integração com APIs de faturação.

Reflexão

O projeto proporcionou uma experiência valiosa em desenvolvimento de software, destacando a importância de rever decisões técnicas cedo — a mudança de Tkinter/MySQL para Flask/SQLite a meio do desenvolvimento foi a decisão mais significativa do projeto, e resultou numa solução que resolve verdadeiramente o problema do cliente, não só o do lojista. Ficou também claro o valor de testar continuamente, em vez de deixar os testes só para o fim.

10. Anexos

- Levantamento de Requisitos (Requisitos_NEXTPOINT_FS.pdf)
- Documento de Entidades, Ações e Relacionamentos (workflow_entidades_flavio_saldanha.pdf)
- Product Backlog (product_backlog_Flavio_Saldanha.xlsx / .pdf)
- Daily Sprint Plan e Daily Scrum Report
- Código-fonte (repositório GitHub)

Este relatório fornece uma visão abrangente do processo de desenvolvimento do projeto Next Point, documentando todas as fases desde o levantamento de requisitos até à implantação e suporte.